Guardrails propuestos — Copiloto Devoluciones

Documento de referencia con los guardrails evaluados para esta rama (`feature/fixed_guardRails`). Para cada uno se indica **qué archivo/función se toca**, **por qué hace falta** (el problema actual) y **qué soluciona**. Ninguno está implementado todavía — este documento es la base para priorizar e implementar.

---

1. Entrada

1.1 Validar formato de `order_number` antes de consultar BD

- **Dónde se toca**: `backend/agents/nodes/triage.py` , función `fetch_order_data` (antes de la llamada a `get_order_details` , ~línea 427).
- **Por qué**: el LLM de `triage_agent` extrae `order_number` de texto libre del cliente (prompt en `TRIAGE_SYSTEM_PROMPT` , línea 60, formato esperado `ORD-2026-00177`), pero nada valida que lo extraído cumpla ese formato antes de usarlo. Si el LLM extrae mal, se dispara una consulta MCP/BD innecesaria que solo falla con un `except Exception` genérico (línea 466).
- **Qué soluciona**: añadir un `re.match(r"^ORD-\d{4}-\d+$", order_number)` como guardia previa evita consultas perdidas y permite devolver un mensaje de error específico ("el formato no es válido") en vez del genérico "no pude encontrar el pedido".

1.2 Detectar intentos de jailbreak / mensajes fuera de tema

- **Dónde se toca**: `backend/agents/nodes/triage.py` , función `triage_agent` , antes de invocar `llm_structured.ainvoke(llm_messages)` (~línea 157).
- **Por qué**: `TRIAGE_SYSTEM_PROMPT` (línea 114) ya instruye "si el problema no es una devolución, indícalo", pero es una instrucción textual dentro del prompt: el modelo puede saltársela con suficiente insistencia ("ignora tus instrucciones anteriores..."). Al ser el primer nodo del grafo, un jailbreak exitoso aquí contamina el resto del flujo (evaluador, resolución).
- **Qué soluciona**: una comprobación previa (heurística por patrones típicos de jailbreak, o un clasificador/LLM barato) que corte la conversación con una respuesta fija si detecta manipulación, en vez de depender solo de que el modelo principal respete el prompt.

1.3 Rate limiting por thread/IP en el chat

- **Dónde se toca**: `backend/api/routers/chat.py`, endpoints `send_message` y `chat_stream` (o como middleware de FastAPI a nivel de router).
- **Por qué**: el único control de concurrencia hoy es `_thread_locks` (línea 37), que evita ejecuciones *simultáneas* del mismo `thread_id`, pero no limita cuántos mensajes o cuántos hilos nuevos puede generar un mismo origen. Cada mensaje dispara mínimo 2 llamadas a OpenAI (triaje + evaluador) más llamadas MCP a Supabase/RAG, sin ningún techo de coste.
- **Qué soluciona**: limitar peticiones por IP y/o por `thread_id` en una ventana de tiempo evita abuso (spam de mensajes, apertura masiva de hilos) y acota el coste de OpenAI/infraestructura ante tráfico malicioso o descontrolado.

---

2. Salida

2.1 Filtro post-generación: bloquear promesas imposibles

- **Dónde se toca**: `backend/agents/nodes/resolution.py`, funciones `resolution_agent` y `rejection_agent`, justo después de `llm.ainvoke(...)` y antes de meter el mensaje en `Command` / `return` (líneas ~215, ~289, ~364, ~389).
- **Por qué**: el commit `835d33c` ya añadió esta protección a nivel de prompt en `REJECTION_SYSTEM_PROMPT` (líneas 94-101: "NUNCA pidas fotos... NUNCA ofrezcas reparación..."), pero sigue siendo una instrucción textual que el modelo puede ignorar bajo presión del usuario o un contexto atípico.
- **Qué soluciona**: una segunda capa determinista — revisar el texto generado contra una lista de frases prohibidas ("envía una foto", "adjunta", "reparación", "sustituci", "contactará con el fabricante") y, si aparece alguna, sustituir el mensaje por uno genérico predefinido en vez de enviarlo al cliente. Convierte una promesa del prompt en una garantía verificada por código.

2.2 Verificar que el importe de reembolso no dependa solo del LLM

- **Dónde se toca**: `backend/agents/nodes/resolution.py`, función `resolution_agent`, al leer `refund_amount = state.get("refund_amount", 0.0)` (línea 129), antes de usarlo para crear el registro de devolución o el HITL.
- **Por qué**: `policy_evaluator_agent` (`backend/agents/nodes/policy.py`) calcula `refund_amount` por artículo vía salida estructurada del LLM (`PolicyEvaluationOutput`), y `resolution.py` lo toma tal cual sin recomputarlo. Si el LLM se equivoca al restar el `restocking_fee` o alucina un importe mayor al precio real del artículo, ese número entra directo en la BD y en el email al cliente (línea 329).
- **Qué soluciona**: recalcular de forma determinista el máximo teórico (`unit_price × quantity` real desde `order_details`) para los artículos elegibles, y si el `refund_amount` del LLM lo supera, recortarlo o

forzar HITL en vez de confiar ciegamente en el cálculo del modelo. Es el guardrail de mayor impacto económico del sistema.

---

3. Reglas de negocio

3.1 Forzar HITL si hay historial de abuso, no solo por importe

- **Dónde se toca**: `backend/agents/nodes/resolution.py`, función `resolution_agent`, en el cálculo de `needs_admin_approval` (línea 150).
- **Por qué**: hoy solo hay dos caminos a `hitl_admin`: `refund_amount > 100€` o el flag `needs_admin_approval`, que a su vez decide el propio LLM del evaluador. El prompt de `policy.py` (línea 48) le pide "ser más estricto" si el cliente tiene más de 5 devoluciones aprobadas en 6 meses, pero es otra instrucción soft, no una regla de código.
- **Qué soluciona**: leer `return_history` (ya disponible en el estado, viene de `get_customer_return_history`) y forzar `needs_admin_approval = True` en código determinista si `return_history.approved >= N` en el periodo, sin depender de que el LLM haya decidido aplicarlo.

3.2 Validar el email del cliente antes de notificar

- **Dónde se toca**: `backend/agents/nodes/resolution.py`, función `resolution_agent`, antes de la llamada a `send_customer_notification` (línea 338).
- **Por qué**: `customer_email` se usa tal cual viene de `order_details["order"]["customer"]["email"]` (línea 432, en `triage.py`), sin validación de formato. Si ese campo llegara a rellenarse desde una fuente menos controlada (formulario público, integración externa), sería un vector de abuso (spam relay, inyección de cabeceras si contiene saltos de línea).
- **Qué soluciona**: validar el formato del email antes de la llamada MCP de notificación, y no ejecutar el envío si no pasa la validación.

---

5. Coste

5.1 Limitar/podar el historial de mensajes en conversaciones largas

- **Dónde se toca**: `backend/agents/nodes/triage.py` (línea 155, `llm_messages = [*context, *state.get("messages", [])]`) y de forma análoga en `backend/agents/nodes/policy.py` (línea 176, donde también se incluye `state.get("messages", [])` en el contexto del evaluador).

- **Por qué**: ningún nodo recorta `state["messages"]`. Cada turno reenvía toda la conversación acumulada al LLM. En una conversación de 4-5 turnos no importa, pero si un cliente indeciso corrige varias veces el estado del artículo o cambia de pedido, el contexto crece sin límite: cada llamada siguiente cuesta más tokens y tarda más, sin ningún techo.
- **Qué soluciona**: recortar o resumir el historial a partir de cierto número de turnos (por ejemplo, quedarse con los últimos N mensajes más un resumen de lo anterior) acota el coste y la latencia por turno de forma predecible, independientemente de cuánto se alargue la conversación.

Resumen de archivos afectados

Guardrail	Archivo	Función
1.1 Formato order_number	`backend/agents/nodes/triage.py`	`fetch_order_data`
1.2 Jailbreak / off-topic	`backend/agents/nodes/triage.py`	`triage_agent`
1.3 Rate limiting	`backend/api/routers/chat.py`	`send_message`, `chat_stream`
2.1 Filtro de promesas	`backend/agents/nodes/resolution.py`	`resolution_agent`, `rejection_agent`
2.2 Consistencia de reembolso	`backend/agents/nodes/resolution.py`	`resolution_agent`
3.1 HITL por historial de abuso	`backend/agents/nodes/resolution.py`	`resolution_agent`
3.2 Validación de email	`backend/agents/nodes/resolution.py`	`resolution_agent`
5.1 Poda de historial	`backend/agents/nodes/triage.py`, `backend/agents/nodes/policy.py`	`triage_agent`, `policy_evaluator_agent`